

Information Retrieval

Final Project

1. Project Overview

The objective is to design and implement a functional search engine capable of processing both English and Arabic text. The system moves beyond simple text-matching by utilizing a Positional Inverted Index for proximity searching and a Vector Space Model (TF-IDF) to rank results by relevance.

2. Document Corpus & Multi-language Support

- Corpus: Use a dataset consisting of at least 10 English documents and 10 Arabic documents, with a total count of no less than 10,000 words.
- Encoding: The system must use UTF-8 encoding to correctly process and display Arabic script.

3. Linguistic Pre-processing Pipeline

The engine must route queries through the appropriate pipeline based on language:

A. English Pipeline

- Tokenization: Extract individual terms from text.
- Stop-word Removal: Filter out high-frequency, low-meaning words (e.g., "the", "is").
- Stemming: Implement the Porter Stemmer to map word variations to their root forms.

B. Arabic Pipeline

- Normalization: Unify Arabic character forms (e.g., Alif shapes $\bar{ا}$, $ا$, $أ$ to $ا$, and $ى$ to $ي$) and remove diacritics (Tashkeel).
- Light Stemming: Implement an Arabic-specific stemmer (e.g., ISRI Stemmer) to handle prefixes and suffixes (e.g., mapping "الجامعات" to "جامع").
- Arabic Stop-words: Use a dedicated Arabic stop-list (e.g., "من", "على", "في").

4. Indexing & Query Capabilities

A. Positional Inverted Index

- Build an index where every term maps to a list of Document IDs and the exact integer positions of each occurrence.

B. Supported Query Types

- Single & Multiple-word Queries: Retrieve documents containing the terms.
- Phrase Queries: Use the positional index to find exact word sequences.
- Proximity Search: Support the /k operator (e.g., employment /3 place) to find terms within \$k\$ words of each other
- Wildcard Queries: Implement a k-gram index (e.g., 2-grams or 3-grams) to support trailing wildcards like comput* or جامع*.

5. Advanced Search Logic

- Spelling Correction: If a term is not in the dictionary, use Edit Distance (Levenshtein) and Jaccard Similarity via the k-gram index to provide "Did you mean?" suggestions.
- Ranked Retrieval (Vector Space Model):
 - TF-IDF Weighting: for each term-document pair
 - Cosine Similarity: Calculate the similarity between the query vector and document vectors. Results must be sorted from highest to lowest score.

6. Evaluation & Performance

The project documentation must include:

- Precision and Recall: Test the system against 3–5 sample queries (including at least one Arabic query) to calculate accuracy.
- Speed: Record the time taken to build the bilingual index versus the time taken to retrieve results.